their browser at the right URL and can use your software. To do this, your program needs to be adapted to run on a web server. This requires a bit of work, but using a single environment also has a distinct advantage: you are in full control over what version of Python gets run.

If you already have some experience with Python, a great answer to building web applications is Django. Django is a third-party Python framework that lets you write code that runs on a web server. Using Django, you write Python code, called a *view*, that is tied to a URL. When your users visit the URL, the Django view runs and returns results to the user's browser. Django does a lot more than that though—it includes tools for doing the following:

- Routing and managing URLs
- Encapsulating what code gets run per page visit
- Reading and writing to databases
- Composing HTML output based on reusable chunks
- Writing multiuser-enabled sites
- Managing user authentication and authorization to your site
- Web-based administrative tools that manage all these features

Django started out as a tool for writing newspaper articles at the *Lawrence Journal-World* in Lawrence, Kansas. Rather than having each article be a single file on a web server, the newspaper wanted a way to reuse pieces of HTML. You don't want to write the newspaper's banner in every file—you want to compose it like you would with code. You also probably don't want your reporters writing HTML; it isn't their expertise.

Django evolved as both a coding framework and a series of tools to manage web content. Among other things, the framework provides the ability to compose HTML out of reusable pieces solving the banner problem, and one of the tools it includes is a web-based interface for the creation of content. Reporters can write articles without worrying about HTML. Django's tools are built on top of the framework, giving you the power to customize them to suit your needs.

The robustness of the Django framework has led to widespread industry adoption. Reddit, YouTube, *The Onion*, Pinterest, Netflix, Dropbox, and Spotify are just a few of the organizations that use Django. Sites like these need scale, and Django has proven itself. Whether you want to build something small or hope to become a massive site, Django can help you. It is a mature platform with a vibrant community, focused on maintaining and evolving a well-tested, secure framework. To top it all off, the original developers at the *Journal-World* made it open source, meaning it is freely available for you to both use and modify for your needs.

1.1 Django's parts

Let's start with a high-level view of what makes up Django, knowing that each of these topics is covered in detail later in the book. One way to understand Django is to think about it in three parts: